A Laboratory Manual for

SQL Lab (ITL302)

Semester-III

Bachelors Degree in Information Technology

The Bombay Salesian Society's

Don Bosco Institute of Technology, Mumbai

(An Autonomous Institute Affiliated to the University of Mumbai)

Department of Information Technology

(AY 2025-26)

DON BOSCO INSTITUTE OF TECHNOLOGY

PREMIER AUTOMOBILES ROAD, KURLA (W.), MUMBAI 400 070.

SUBJECT DBMS BRANCH/DIV. IT

SEMESTER III ROLL NO. 57

INDEX

Sr. No.	Title of Experiment/Tutorial/Assignment	Page No.	Performance Date	Submission Date	Grade/ Remarks	Sign.
1.	E-R Diagram		27/7/25	1/12/25		
2	DDL Commands		25/7/25	1/12/25		
3	DML Commands		4/8/25	1/12/25		
4.	Aggregate functions and Group by with/without having		11/8/25	1/12/25		
5.	SQL Joins		1/9/25	1/12/25		
6.	SQL Trigger		15/9/25	1/12/25		
7.	SQL Stored Procedure.		6/10/25	1/12/25		
8.	SQL Views		25/10/25	1/12/25		
9.	JDBC Programming		15/10/25	1/12/25		
10.	SQL Subquery		27/10/25	1/12/25		
11	TCL Commands		3/11/25	1/12/25		

CERTIFICATE

This is to certify that Mr./~~Ms.~~ Ritesh . B. Singh Roll No. 57

has completed the specified term-work in the subject of DBMS

in a satisfactory manner in the college during Academic Year 2025 - 2026

Date : 1/12/2025

Lecturer in charge

Scanned with OKEN Scanner

The Bombay Salesian Society's

Don Bosco Institute of Technology, Mumbai

Deptartment of Information Technology

Experiment – COs Mapping Table

S.No	Title of Experiment	Date of Performance	Date of Submission	Grade/Marks	CO	Remarks
1	ER- Diagram					
2	DDL Commands (Create,Alter,Drop& Trucate)					
3	DML Commands with SQL Operators (Insert,Update,Delete & Select)					
4	Aggregate functions and GROUP By with/without HAVING					
5	SQL JOINs					
6	SQL Trigger					
7	SQL stored procedure					
8	SQL VIEWs					
9	JDBC Programming					
10	SQL Subquery					
11	TCL Commands					

Student Name and Rollno

Subject In-charge

The Bombay Salesian Society's

Don Bosco Institute of Technology, Mumbai

(An Autonomous Institute Affiliated to the University of Mumbai)

Department. of Information Technology

SQL LAB **57_Ritesh_S**

A.Y.2025-26 **SEM III**

This is to certify that Mr**. Ritesh Singh** has successfully completed the Termwork in the subject of **SQL LAB** during **S.E Semester III** in the Academic Year **2025-2026.**

Faculty Incharge

(Prof. Shiv Negi)

EXPERIMENT 1

AIM: To study and construct an Entity–Relationship (ER) Diagram for an insurance management system involving customers, cars, insurance companies, and accident records by identifying the entities, attributes, and relationships among them.

THEORY: An Entity–Relationship (ER) Diagram is a visual representation of real-world data and how different objects in a system are related to each other. It is commonly used during database design to understand the structure of data before creating actual tables.

1. Identifying Entities

Entities are objects that hold information. Based on the problem statement, the following entities were identified:

Customer

Car

Insurance Company

Accidents

Each entity represents an independent conceptual object in the system.

2. Identifying Attributes

Attributes describe the properties of each entity. The attributes used in the ER model are:

Customer: Cust_id, Cust_name

Car: Car_number, Car_model, Owner_name

Insurance Company: Company_id, Company_name, Com_Address

Accidents: Acc_id, Car_number (as reference)

Attributes help in uniquely identifying and describing each real-world object.

3. Identifying Relationships

A relationship represents how two entities are connected. Based on the system requirements, the following relationships were established:

Customer – Owns – Car

A single customer can own multiple cars.

Cardinality: 1 : M

Customer – Registers For – Insurance Company

One insurance company can have many registered customers.

Cardinality: 1 : M

Car – Records – Accident

A car may have zero or multiple accident records.

Cardinality: 1 : N

These relationships help define how data flows and interacts within the system.

4. Constructing the ER Diagram

The ER diagram is drawn by combining all identified entities, their attributes, and relationships.

Each relationship is represented using a diamond shape, entities using rectangles, and attributes using ovals.

Primary keys are underlined or highlighted to show uniqueness.

The completed ER diagram represents the full structure of the insurance system and becomes the foundation for designing the relational database.

EXECUTION:

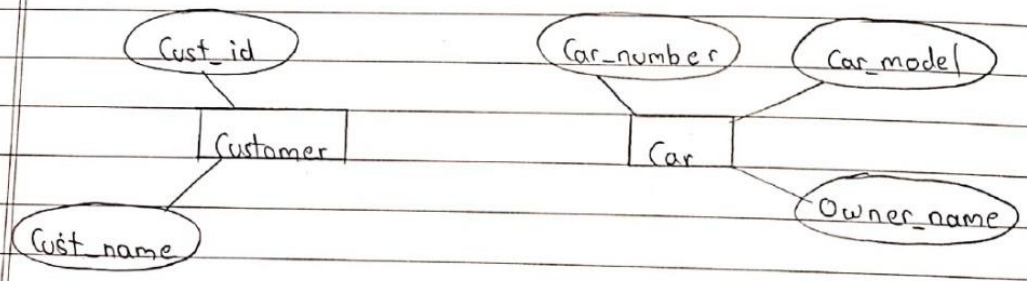
Date. : | | Page No. :

Step 1 :- Identifying Entities

i> Customer
ii> Car
iii> Insaurance. Company
iv> Accidents

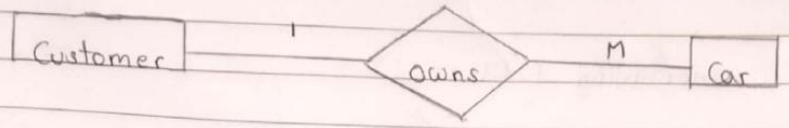
Step 2 :- Identify Attributes

i> Customer (Cust_id, Cust_name)
ii> Car (Car_number, car_model, Owner_name)
iii> Insaurance Company (Company_Id, Company_name, Com_Address)
iv> Accidents (Acc_id, Car)



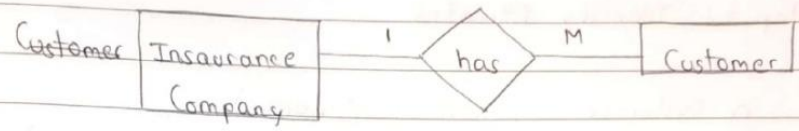
Step 3 :- Identify Relationship.

Scanned with OKEN Scanner

1> 

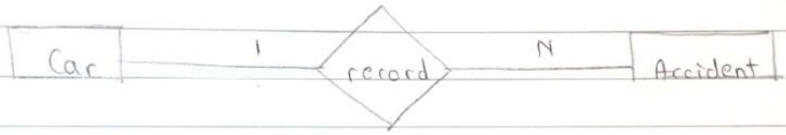
A signle customer can own multiple cars.

2>

2> 

A insurance company has many customers.

3>



Each car is associated with zero or any number of accidents.

Step 4:- Constructing ER diagram by merging all above relationships.

Scanned with OKEN Scanner

Date. :

Page No. :

Teacher's Sign

Scanned with OKEN Scanner

CONCLUSION: In this practical, an ER diagram was successfully constructed for an insurance management system by identifying the main entities—Customer, Car, Insurance Company, and Accidents—and determining their attributes and relationships.

The model clearly shows how customers own cars, how they register with insurance companies, and how cars may be involved in multiple accidents.

This exercise helped in understanding the importance of ER diagrams as a planning tool for database design. It also demonstrated how visual modelling simplifies the process of organizing data and defining relationships before creating database tables.

EXPERIMENT 2

AIM: To study and execute SQL Data Definition Language (DDL) commands such as CREATE, ALTER, DROP and TRUNCATE for defining and modifying the structure of database tables.

THEORY: Data Definition Language (DDL) is a core part of SQL used to define, organize, and manage the structure of tables and other objects in a database. Unlike DML, which manipulates the actual data stored in tables, DDL commands work on the *schema level*, meaning they change how the database itself is designed. Every time a database table, column, or constraint is created or modified, DDL commands are involved.

DDL commands are automatically committed in SQL, which means changes made by them cannot be rolled back. These commands ensure that the database structure remains consistent, well-organized, and optimized for storing and retrieving data.

The primary DDL commands include:

1. CREATE

This command is used to create new database objects such as databases, tables, views, and indexes.
It defines the structure by specifying column names, data types, constraints, and relationships.

Syntax:

```
CREATE TABLE table_name (
  column1 datatype constraint,
  column2 datatype constraint,
  ...
);
```

2. ALTER

ALTER is used to modify the structure of an existing table.
It allows users to add new columns, delete unwanted columns, change data types,

rename columns, or modify constraints.
ALTER gives flexibility to adjust the table design without deleting existing data.

Syntax:

ALTER TABLE table_name ADD column_name datatype;

ALTER TABLE table_name MODIFY column_name new_datatype;

ALTER TABLE table_name DROP column_name;

3. DROP

DROP permanently removes a database object such as a table, database, or view.
Once a table is dropped, all its data and structure are lost.
This command must be used carefully, as it cannot be undone.

Syntax:

DROP TABLE table_name;

4. TRUNCATE

TRUNCATE removes all records from a table instantly while keeping the table structure intact.
Unlike DELETE (a DML command), TRUNCATE is faster and does not generate individual row logs.
It is useful when large amounts of data must be cleared quickly.

Syntax:

TRUNCATE TABLE table_name;

Importance of DDL:

- It provides a clear structure for storing data.
- Ensures data consistency by enforcing constraints like PRIMARY KEY and FOREIGN KEY.
- Makes the database scalable by allowing structural modifications.
- Forms the foundation on which DML operations are performed later.

EXECUTION:

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin>mysql -u root -p
Enter password: ******
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
-----------------------------------------CREATE----------------------------------------------
mysql> create Database CollegeDB;
Query OK, 1 row affected (0.01 sec)

mysql> show databases;
+ ----------------- +
| Database          |
+ ----------------- +
| collegedb         |
| information_schema |
```

```
| mysql              |
| performance_schema |
| sys                |
+ ----------------- +
5 rows in set (0.00 sec)

mysql> use CollegeDB;
Database changed
mysql> Create Table StudentInfo (StudID int(3) ,Sname Varchar(20), Age int(2),
Department varchar(20));
Query OK, 0 rows affected, 2 warnings (0.04 sec)

mysql> desc table StdentInfo;
ERROR 1146 (42S02): Table 'collegedb.stdentinfo' doesn't exist
mysql> desc table StudentInfo;
+----+-------------+-------------+------------+------+---------------+------+---------+------+------+-----
-----+ ------ +
| id | select_type | table       | partitions | type | possible_keys | key  | key_len | ref
| rows | filtered | Extra |
+----+-------------+-------------+------------+------+---------------+------+---------+------+------+-----
-----+ ------ +
|  1 | SIMPLE      | StudentInfo | NULL       | ALL  | NULL          | NULL | NULL    | NULL |
1 |   100.00 | NULL  |
+----+-------------+-------------+------------+------+---------------+------+---------+------+------+-----
-----+ ------ +
1 row in set, 1 warning (0.01 sec)

mysql> select * from StudentInfo;
Empty set (0.00 sec)
```

```
mysql> insert into StudentInfo Values(101, 'Roger',20,IT);
ERROR 1054 (42S22): Unknown column 'IT' in 'field list'
mysql> insert into StudentInfo Values(101, 'Roger',20,'IT');
Query OK, 1 row affected (0.01 sec)

mysql> select * from StudentInfo;
+--------+-------+------+ -----------+
| StudID | Sname | Age  | Department |
+--------+-------+------+ -----------+
|    101 | Roger |   20 | IT         |
+--------+-------+------+ -----------+
1 row in set (0.00 sec)
-----------------------------------------ALTER  ---------------------------------------------

mysql> Alter table StudentInfo Add Email Varchar(30);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc StudentInfo;
+------------+-------------+------+-----+---------+------- +
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+------- +
| StudID     | int         | YES  |     | NULL    |       |
| Sname      | varchar(20) | YES  |     | NULL    |       |
| Age        | int         | YES  |     | NULL    |       |
| Department | varchar(20) | YES  |     | NULL    |       |
| Email      | varchar(30) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+------- +
```

5 rows in set (0.02 sec)

```
mysql> alter table StudentInfo modify Age int NOT NULL;
Query OK, 0 rows affected (0.12 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> Alter Table StudentInfo change Column Sname FullName Varchar(50);
Query OK, 0 rows affected (0.22 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc StudentInfo;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| StudID     | int         | YES  |     | NULL    |       |
| FullName   | varchar(50) | YES  |     | NULL    |       |
| Age        | int         | NO   |     | NULL    |       |
| Department | varchar(20) | YES  |     | NULL    |       |
| Email      | varchar(30) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
5 rows in set (0.00 sec)

mysql> select * from StudentInfo;
+--------+----------+-----+------------+------+
| StudID | FullName | Age | Department | Email |
+--------+----------+-----+------------+------+
|    101 | Roger    |  20 | IT         | NULL |
+--------+----------+-----+------------+------+
```

1 row in set (0.00 sec)

----------TRUNCATE ----------

mysql> Truncate Table StudentInfo;

Query OK, 0 rows affected (0.04 sec)

mysql> desc StudentInfo;

```
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| StudID     | int         | YES  |     | NULL    |       |
| FullName   | varchar(50) | YES  |     | NULL    |       |
| Age        | int         | NO   |     | NULL    |       |
| Department | varchar(20) | YES  |     | NULL    |       |
| Email      | varchar(30) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
```

5 rows in set (0.00 sec)

mysql> select * from StudentInfo;

Empty set (0.00 sec)

----------DROP----------

mysql> Drop Table StudentInfo;

Query OK, 0 rows affected (0.02 sec)

mysql> desc StudentInfo;

ERROR 1146 (42S02): Table 'collegedb.studentinfo' doesn't exist

mysql>

CONCLUSION: In this experiment, DDL commands were used to create, alter, modify, and manage the structure of database tables. These commands define how data is stored, organized, and controlled within the database. Understanding DDL is essential because it forms the foundation of every SQL-based application. Its commands ensure proper table design, data integrity, and efficient database management.

EXPERIMENT 3

AIM: To study and execute SQL Data Manipulation Language (DML) commands such as INSERT, UPDATE, DELETE, and SELECT to manipulate and retrieve data stored in a database table.

THEORY: Data Manipulation Language (DML) is an essential component of SQL used for handling the data stored inside relational database tables. Unlike DDL commands that define the structure of the database, DML commands operate directly on the table records. They enable users to add new data, modify existing records, remove unwanted information, and fetch data as per requirements.

The main DML commands are:

1. INSERT:

Used to add new rows into a table. It allows inserting values for all or selected columns.
This command is used during initial data population or when new records are created during operations.

Example:

```
INSERT INTO Student(RollNo, Name, Marks, Branch) VALUES(1,'Amit',85,'Computer');
```

2. UPDATE:

Used to modify existing records.
It can update one or more columns but usually requires a WHERE clause to ensure only specific rows are changed.

Example:

```
UPDATE Student SET Marks = 95 WHERE RollNo = 1;
```

3. DELETE:

Used to remove records from a table.
A WHERE clause is used to delete selected rows; without it, all records get deleted.

Example:

```
DELETE FROM Student WHERE RollNo = 2;
```

4. SELECT:

Used to retrieve data from database tables.
SELECT supports filtering, sorting, grouping, joining, and aggregate operations.
It is the most frequently used SQL command for viewing data.

Example:

SELECT * FROM Student;

DML commands are crucial for maintaining data accuracy, performing everyday transactions, and enabling meaningful data extraction in real-world applications such as banking, inventory systems, billing software, and student management systems.

EXECUTION:

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin>mysql -u root -p
Enter password: ******
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use CollegeDB;
Database changed
```

```
mysql> show tables;
Empty set (0.02 sec)

mysql> create table StudentInfo ( StudID int(3), Sname varchar(30), Age int(2),
Department Varchar(10), city varchar(10));
Query OK, 0 rows affected, 2 warnings (0.03 sec)

mysql> desc StudentInfo;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| StudID     | int         | YES  |     | NULL    |       |
| Sname      | varchar(30) | YES  |     | NULL    |       |
| Age        | int         | YES  |     | NULL    |       |
| Department | varchar(10) | YES  |     | NULL    |       |
| city       | varchar(10) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
5 rows in set (0.00 sec)
----------------------------------------INSERT----------------------------------------------
mysql> insert into StudentInfo Values(101,'Henry',20,'IT','Mumbai');
Query OK, 1 row affected (0.01 sec)

mysql> insert into StudentInfo Values(102,'Maverick',20,'IT','Mumbai');
Query OK, 1 row affected (0.01 sec)

mysql> insert into StudentInfo Values(103,'Harsh',19,'COMPS','Delhi');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> insert into StudentInfo Values(104,'Sameer',19,'EXTC','Mumbai');
Query OK, 1 row affected (0.01 sec)

mysql> insert into StudentInfo Values(105,'Rohan',20,'MECH','Mumbai');
Query OK, 1 row affected (0.01 sec)

mysql> Select * from StudentInfo;
+--------+----------+------+------------+ ------- +
| StudID | Sname    | Age  | Department | city   |
+--------+----------+------+------------+ ------- +
|    101 | Henry    |   20 | IT         | Mumbai |
|    102 | Maverick |   20 | IT         | Mumbai |
|    103 | Harsh    |   19 | COMPS      | Delhi  |
|    104 | Sameer   |   19 | EXTC       | Mumbai |
|    105 | Rohan    |   20 | MECH       | Mumbai |
+--------+----------+------+------------+ ------- +
5 rows in set (0.00 sec)

mysql> Select StudID,Sname from StudentInfo Where city = 'Mumbai';
+--------+---------+
| StudID | Sname   |
+--------+---------+
|    101 | Henry   |
|    102 | Maverick |
|    104 | Sameer  |
|    105 | Rohan   |
+--------+---------+
4 rows in set (0.00 sec)
```

----------UPDATE----------

```
mysql> update StudentInfo Set Age=20 where Sname='Harsh';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> Select * from StudentInfo;
+--------+----------+------+------------+ ------- +
| StudID | Sname    | Age  | Department | city   |
+--------+----------+------+------------+ ------- +
|    101 | Henry    |   20 | IT         | Mumbai |
|    102 | Maverick |   20 | IT         | Mumbai |
|    103 | Harsh    |   20 | COMPS      | Delhi  |
|    104 | Sameer   |   19 | EXTC       | Mumbai |
|    105 | Rohan    |   20 | MECH       | Mumbai |
+--------+----------+------+------------+ ------- +
5 rows in set (0.00 sec)
```

----------DELETE----------

```
mysql> delete from StudentInfo Where Department='MECH';
Query OK, 1 row affected (0.01 sec)

mysql> Select * from StudentInfo;
+--------+----------+------+------------+ ------- +
| StudID | Sname    | Age  | Department | city   |
+--------+----------+------+------------+ ------- +
|    101 | Henry    |   20 | IT         | Mumbai |
|    102 | Maverick |   20 | IT         | Mumbai |
```

```
|    103 | Harsh     |   20 | COMPS       | Delhi  |
|    104 | Sameer    |   19 | EXTC        | Mumbai |
+--------+----------+------+------------+ ------- +
4 rows in set (0.00 sec)

mysql>
```

CONCLUSION: In this experiment, various SQL DML commands were successfully executed on a database table.
Using INSERT, new records were added; UPDATE allowed modification of existing data; DELETE removed unnecessary rows; and SELECT enabled retrieval of information based on different conditions.
This experiment demonstrates how DML commands form the foundation of daily database operations and are essential for managing and maintaining data efficiently in real-world applications.

EXPERIMENT 4

AIM: To study and execute SQL aggregate functions such as SUM, AVG, COUNT, MIN, MAX and to apply GROUP BY and HAVING clauses for grouping and filtering aggregated data.

THEORY: Aggregate functions in SQL are used to perform calculations on a set of values and return a single summarized result. They are commonly used in reporting, analysis, and decision-making. Aggregate functions work on numeric as well as non-numeric data (depending on the function) and are often used along with the GROUP BY clause.

The GROUP BY clause groups rows that have the same values in specified columns. It is used when aggregate calculations are required for different categories or groups such as departments, branches, locations, or product types.

The HAVING clause is similar to the WHERE clause but is used specifically with aggregated results. Because WHERE works on individual rows and HAVING works on groups, HAVING is essential when filtering grouped values like total salary, average marks, or employee count.

Common Aggregate Functions:

- SUM()
  Adds values of a numeric column.

  SELECT SUM(Salary) FROM Employee;

- AVG()
  Returns the average value of a numeric column.

  SELECT AVG(Salary) FROM Employee;

- COUNT()
  Returns the number of rows or non-NULL values.

  SELECT COUNT(*) FROM Employee;

- MIN()
  Returns the smallest value in a column.

  SELECT MIN(Salary);

- MAX()
  Returns the largest value in a column.

  MAX(Salary) FROM Employee;

GROUP BY:

Used to divide table data into groups and apply aggregate functions to each group.

HAVING:

Used to filter aggregated data after grouping.
WHERE filters rows — HAVING filters groups.

These features are essential in analyzing data such as employee salary structures, student performance, sales summaries, and inventory reports.

EXECUTION:

Microsoft Windows [Version 10.0.19045.6466]

(c) Microsoft Corporation. All rights reserved.

C:\Users\admin>mysql -u root -p

Enter password: ******

Welcome to the MySQL monitor. Commands end with ; or \g.

Your MySQL connection id is 14

Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its

affiliates. Other names may be trademarks of their respective

owners.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show Databases;
+ ----------------- +
| Database          |
+ ----------------- +
| collegedb         |
| information_schema |
| mysql             |
| performance_schema |
| sys               |
+ ----------------- +
5 rows in set (0.00 sec)

mysql> create database Employee;
Query OK, 1 row affected (0.01 sec)

mysql> use Employee;
Database changed
mysql> create table EmpInfo (EmpID int, EmpName Varchar(20), Dept Varchar(10),
Salary int, city varchar(20));
Query OK, 0 rows affected (0.03 sec)

mysql> desc Employee;
ERROR 1146 (42S02): Table 'employee.employee' doesn't exist
mysql> desc EmpInfo;
+---------+-------------+------+-----+---------+ ------ +
```

```
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+ ------ +
| EmpID   | int         | YES  |     | NULL    |       |
| EmpName | varchar(20) | YES  |     | NULL    |       |
| Dept    | varchar(10) | YES  |     | NULL    |       |
| Salary  | int         | YES  |     | NULL    |       |
| city    | varchar(20) | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+ ------ +
5 rows in set (0.00 sec)
```

```
mysql> Insert into EmpInfo(1001,'Rohan','IT',45000,'Delhi');
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '1001,'Rohan','IT',45000,'Delhi')' at line 1
mysql> Insert into EmpInfo(1,'Rohan','IT',45000,'Delhi');
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '1,'Rohan','IT',45000,'Delhi')' at line 1
mysql> Insert into EmpInfo values(1,'Rohan','IT',45000,'Delhi');
Query OK, 1 row affected (0.01 sec)

mysql> Insert into EmpInfo values(2,'Simran','HR',35000,'Mumbai');
Query OK, 1 row affected (0.01 sec)

mysql> Insert into EmpInfo values(3,'Amit','IT',60000,'Delhi');
Query OK, 1 row affected (0.01 sec)

mysql> Insert into EmpInfo values(4,'Priya','Finance',55000,'Pune');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> Insert into EmpInfo values(5,'Karan','Finance',70000,'Delhi');
Query OK, 1 row affected (0.01 sec)

mysql> Insert into EmpInfo values(6,'Sara','IT',30000,'Mumbai');
Query OK, 1 row affected (0.01 sec)

mysql> select * from EmpInfo;
+-------+---------+---------+--------+------- +
| EmpID | EmpName | Dept    | Salary | city   |
+-------+---------+---------+--------+------- +
|     1 | Rohan   | IT      |  45000 | Delhi  |
|     2 | Simran  | HR      |  35000 | Mumbai |
|     3 | Amit    | IT      |  60000 | Delhi  |
|     4 | Priya   | Finance |  55000 | Pune   |
|     5 | Karan   | Finance |  70000 | Delhi  |
|     6 | Sara    | IT      |  30000 | Mumbai |
+-------+---------+---------+--------+------- +
6 rows in set (0.00 sec)
----------------------------AGGREGATE  FUNCTIONS--------------------------

mysql> select count(*) AS TotalEmployees from EmpInfo;
+ ------------- +
| TotalEmployees |
+ ------------- +
|             6 |
+ ------------- +
1 row in set (0.01 sec)
```

```
mysql> select Avg(*) AS AvgSalary from EmpInfo;

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '*) AS AvgSalary from EmpInfo' at line 1

mysql> select Avg(Salary) AS AvgSalary from EmpInfo;

+ ----------+

| AvgSalary  |

+ ----------+

| 49166.6667 |

+ ----------+

1 row in set (0.01 sec)


mysql> select Min(Salary) AS MinSalary,Max(Salary) AS MaxSalary from EmpInfo;

+-----------+ --------- +

| MinSalary | MaxSalary |

+-----------+ --------- +

|     30000 |     70000 |

+-----------+ --------- +

1 row in set (0.01 sec)


mysql> select Sum(Salary) AS TotalSalary from EmpInfo;

+ -----------+

| TotalSalary |

+ -----------+

|      295000 |

+ -----------+

1 row in set (0.00 sec)
```

----------GROUP BY----------

```
mysql> select Dept, Avg(Salary) AS avgSalary from EmpInfo group by Dept;
+---------+-----------+
| Dept    | avgSalary |
+---------+-----------+
| IT      | 45000.0000 |
| HR      | 35000.0000 |
| Finance | 62500.0000 |
+---------+-----------+
3 rows in set (0.00 sec)
```

```
mysql> select city, Avg(Salary) AS AvgCitySalary from EmpInfo group by city having
Avg(Salary)>45000;
+-------+------------- +
| city  | AvgCitySalary |
+-------+------------- +
| Delhi |    58333.3333 |
| Pune  |    55000.0000 |
+-------+------------- +
2 rows in set (0.00 sec)
```

```
mysql>
```

CONCLUSION: In this experiment, aggregate functions such as SUM, AVG, COUNT, MIN, and MAX were applied to summarize and analyze data from a table. The GROUP BY clause was used to categorize records into logical groups, and the HAVING clause filtered these grouped results based on specific conditions. This experiment demonstrates how SQL aggregation is essential for producing reports, comparisons, summaries, and analytical insights in real-world database applications.

EXPERIMENT 5

AIM: To study and implement different types of SQL JOIN operations such as INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN and CROSS JOIN, in order to retrieve related data from two or more tables based on common fields

THEORY:

Joins in SQL are used to combine data from two or more tables based on a related column between them. They play a fundamental role in relational database systems, allowing meaningful retrieval of information that is distributed across multiple tables. By linking rows through common attributes, joins enable efficient querying, reporting, and analysis while maintaining normalized database design.

1. CROSS JOIN (Cartesian Product)

A CROSS JOIN returns the Cartesian product of two tables. Every row of the first table is combined with every row of the second table.

- Use case: Generating all possible combinations or when a pairing of every row is explicitly required.
- Result size: rows(A) × rows(B)
- Note: This type of join does not require a join condition.

2. INNER JOIN

An INNER JOIN retrieves only the rows that have matching values in both tables according to the join condition specified.

- Use case: When data must exist in both tables to be included in the result.
- Behavior: Non-matching rows from either side are excluded.
- Syntax: ... FROM A INNER JOIN B ON A.key = B.key

3. LEFT OUTER JOIN

A LEFT JOIN returns all rows from the left table and only matching rows from the right table. Non-matching rows from the right table are represented with NULL values.

- Use case: When all records from the primary (left) table must be preserved.
- Behavior: Right-side non-matches appear with NULLs.

4. RIGHT OUTER JOIN

A RIGHT JOIN is the mirror of a LEFT JOIN. It returns all rows from the right table and only matching rows from the left table.

- Use case: When all records from the right table need to be included in the final output.
- Behavior: Left-side non-matches appear with NULL values.

5. FULL OUTER JOIN

A FULL OUTER JOIN returns all rows from both tables. Matching rows are displayed together, while non-matching rows from either side appear with NULLs for the opposite table's columns.

EXECUTION:

```
Microsoft Windows [Version 10.0.19045.6466]

(c) Microsoft Corporation. All rights reserved.


C:\Users\admin>mysql -u root -p

Enter password: ******

Welcome to the MySQL monitor.  Commands end with ; or \g.

Your MySQL connection id is 15

Server version: 8.0.44 MySQL Community Server - GPL
```

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> use Employee;
Database changed
mysql> show tables;
+ ----------------- +
| Tables_in_employee |
+ ----------------- +
| empinfo            |
+ ----------------- +
1 row in set (0.01 sec)

mysql> select * from empinfo;
+-------+---------+---------+--------+------- +
| EmpID | EmpName | Dept    | Salary | city   |
+-------+---------+---------+--------+------- +
|     1 | Rohan   | IT      |  45000 | Delhi  |
|     2 | Simran  | HR      |  35000 | Mumbai |
|     3 | Amit    | IT      |  60000 | Delhi  |
|     4 | Priya   | Finance |  55000 | Pune   |
|     5 | Karan   | Finance |  70000 | Delhi  |
```

```
|     6 | Sara     | IT       |  30000 | Mumbai |
+-------+---------+---------+--------+------- +
6 rows in set (0.00 sec)

mysql> create table department(DeptID int, DeptName varchar(20), Manager
varchar(20));
Query OK, 0 rows affected (0.03 sec)

mysql> insert into department values(1,'IT','Mr. Arjun');
Query OK, 1 row affected (0.01 sec)

mysql> insert into department values(2,'HR','Ms. Kavita');
Query OK, 1 row affected (0.01 sec)

mysql> insert into department values(3,'Finance','Mr. Suresh');
Query OK, 1 row affected (0.01 sec)

mysql> insert into department values(4,'Marketing','Ms. Natasha');
Query OK, 1 row affected (0.01 sec)

mysql> insert into department values(5,'Sales','Mr. Raghav');
Query OK, 1 row affected (0.01 sec)

mysql> select * from department;
+--------+-----------+ ----------- +
| DeptID | DeptName  | Manager     |
+--------+-----------+ ----------- +
|      1 | IT        | Mr. Arjun   |
```

```
|     2 | HR        | Ms. Kavita  |
|     3 | Finance   | Mr. Suresh  |
|     4 | Marketing | Ms. Natasha |
|     5 | Sales     | Mr. Raghav  |
+--------+-----------+ ----------- +
5 rows in set (0.00 sec)
```

----------CROSS JOIN----------

```
mysql> select * from empinfo Cross join depertment;
ERROR 1146 (42S02): Table 'employee.depertment' doesn't exist
mysql> select * from empinfo Cross join department;
+-------+---------+---------+--------+--------+--------+-----------+ ------------ +
| EmpID | EmpName | Dept    | Salary | city   | DeptID | DeptName  | Manager     |
+-------+---------+---------+--------+--------+--------+-----------+ ------------ +
|     1 | Rohan   | IT      |  45000 | Delhi  |      5 | Sales     | Mr. Raghav  |
|     1 | Rohan   | IT      |  45000 | Delhi  |      4 | Marketing | Ms. Natasha |
|     1 | Rohan   | IT      |  45000 | Delhi  |      3 | Finance   | Mr. Suresh  |
|     1 | Rohan   | IT      |  45000 | Delhi  |      2 | HR        | Ms. Kavita  |
|     1 | Rohan   | IT      |  45000 | Delhi  |      1 | IT        | Mr. Arjun   |
|     2 | Simran  | HR      |  35000 | Mumbai |      5 | Sales     | Mr. Raghav  |
|     2 | Simran  | HR      |  35000 | Mumbai |      4 | Marketing | Ms. Natasha |
|     2 | Simran  | HR      |  35000 | Mumbai |      3 | Finance   | Mr. Suresh  |
|     2 | Simran  | HR      |  35000 | Mumbai |      2 | HR        | Ms. Kavita  |
|     2 | Simran  | HR      |  35000 | Mumbai |      1 | IT        | Mr. Arjun   |
|     3 | Amit    | IT      |  60000 | Delhi  |      5 | Sales     | Mr. Raghav  |
|     3 | Amit    | IT      |  60000 | Delhi  |      4 | Marketing | Ms. Natasha |
|     3 | Amit    | IT      |  60000 | Delhi  |      3 | Finance   | Mr. Suresh  |
|     3 | Amit    | IT      |  60000 | Delhi  |      2 | HR        | Ms. Kavita  |
|     3 | Amit    | IT      |  60000 | Delhi  |      1 | IT        | Mr. Arjun   |
```

```
|    4 | Priya   | Finance |  55000 | Pune   |     5 | Sales     | Mr. Raghav  |
|    4 | Priya   | Finance |  55000 | Pune   |     4 | Marketing | Ms. Natasha |
|    4 | Priya   | Finance |  55000 | Pune   |     3 | Finance   | Mr. Suresh  |
|    4 | Priya   | Finance |  55000 | Pune   |     2 | HR        | Ms. Kavita  |
|    4 | Priya   | Finance |  55000 | Pune   |     1 | IT        | Mr. Arjun   |
|    5 | Karan   | Finance |  70000 | Delhi  |     5 | Sales     | Mr. Raghav  |
|    5 | Karan   | Finance |  70000 | Delhi  |     4 | Marketing | Ms. Natasha |
|    5 | Karan   | Finance |  70000 | Delhi  |     3 | Finance   | Mr. Suresh  |
|    5 | Karan   | Finance |  70000 | Delhi  |     2 | HR        | Ms. Kavita  |
|    5 | Karan   | Finance |  70000 | Delhi  |     1 | IT        | Mr. Arjun   |
|    6 | Sara    | IT      |  30000 | Mumbai |     5 | Sales     | Mr. Raghav  |
|    6 | Sara    | IT      |  30000 | Mumbai |     4 | Marketing | Ms. Natasha |
|    6 | Sara    | IT      |  30000 | Mumbai |     3 | Finance   | Mr. Suresh  |
|    6 | Sara    | IT      |  30000 | Mumbai |     2 | HR        | Ms. Kavita  |
|    6 | Sara    | IT      |  30000 | Mumbai |     1 | IT        | Mr. Arjun   |
+-------+---------+---------+--------+--------+--------+-----------+ ------------ +
30 rows in set (0.00 sec)
```

----------INNER JOIN----------

```
mysql> select e.EmpName,e.Dept,d.Manager from empinfo e inner join department
d on e.Dept= d.DeptName;
+---------+---------+ ---------- +
| EmpName | Dept    | Manager    |
+---------+---------+ ---------- +
| Rohan   | IT      | Mr. Arjun  |
| Simran  | HR      | Ms. Kavita |
| Amit    | IT      | Mr. Arjun  |
```

| Priya | Finance | Mr. Suresh |

| Karan | Finance | Mr. Suresh |

| Sara | IT | Mr. Arjun |

+---------+---------+ ---------- +

6 rows in set (0.01 sec)

----------LEFT OUTER JOIN----------

mysql> select e.EmpName,e.Dept,d.Manager from empinfo e left join department d on e.Dept= d.DeptName;

+---------+---------+ ---------- +

| EmpName | Dept | Manager |

+---------+---------+ ---------- +

| Rohan | IT | Mr. Arjun |

| Simran | HR | Ms. Kavita |

| Amit | IT | Mr. Arjun |

| Priya | Finance | Mr. Suresh |

| Karan | Finance | Mr. Suresh |

| Sara | IT | Mr. Arjun |

+---------+---------+ ---------- +

6 rows in set (0.00 sec)

----------RIGHT OUTER JOIN----------

mysql> select e.EmpName,e.Dept,d.Manager from empinfo e right join department d on e.Dept= d.DeptName;

+---------+---------+ ---------- +

| EmpName | Dept | Manager |

+---------+---------+ ---------- +

| Sara | IT | Mr. Arjun |

| Amit | IT | Mr. Arjun |

| Rohan | IT | Mr. Arjun |

| Simran | HR | Ms. Kavita |

| Karan | Finance | Mr. Suresh |

| Priya | Finance | Mr. Suresh |

| NULL | NULL | Ms. Natasha |

| NULL | NULL | Mr. Raghav |

+---------+---------+ ---------- +

8 rows in set (0.00 sec)

----------FULL OUTER JOIN----------

mysql> select e.EmpName,e.Dept,d.Manager from empinfo e full outer join department d on e.Dept= d.DeptName;

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'full outer join department d on e.Dept= d.DeptName' at line 1

mysql> select e.EmpName,e.Dept,d.Manager from empinfo e left join department d on e.Dept= d.DeptName union select e.EmpName,e.Dept,d.Manager from empinfo e right join department d on e.Dept = d.DeptName;

+---------+---------+ ---------- +

| EmpName | Dept | Manager |

+---------+---------+ ---------- +

| Rohan | IT | Mr. Arjun |

| Simran | HR | Ms. Kavita |

| Amit | IT | Mr. Arjun |

| Priya | Finance | Mr. Suresh |

| Karan | Finance | Mr. Suresh |

| Sara | IT | Mr. Arjun |

| NULL | NULL | Ms. Natasha |

| NULL | NULL | Mr. Raghav |

+---------+---------+ ---------- +

8 rows in set (0.02 sec)

mysql>

CONCLUSION:

In this experiment, various SQL JOIN operations were successfully implemented to retrieve related data from multiple tables. The INNER JOIN retrieved only matching records, while LEFT and RIGHT OUTER JOINs displayed unmatched rows from one side along with NULL values for missing data The CROSS JOIN generated the Cartesian product of two tables.
Through this experiment, the concept of relational data retrieval was understood, and the role of JOINs in connecting tables using common keys was clearly demonstrated.

EXPERIMENT 6

AIM: To study and implement database triggers in MySQL

THEORY:

A trigger is a stored program in MySQL that automatically executes when a specific event occurs on a table, such as INSERT, UPDATE, or DELETE. Triggers help in automating tasks, maintaining data consistency, validating inputs, and recording changes in the database.

Triggers are used when we need actions to run automatically such as calculating derived values (like total marks), logging changes (old values, deleted rows), enforcing rules before saving data, or maintaining audit logs.

Syntax of a Trigger:

```
CREATE TRIGGER trigger_name
BEFORE | AFTER INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
BEGIN
-- trigger logic
END;
```

Types of Triggers in MySQL:

1. BEFORE INSERT
2. AFTER INSERT
3. BEFORE UPDATE
4. AFTER UPDATE
5. BEFORE DELETE
6. AFTER DELETE

EXECUTION:

Microsoft Windows [Version 10.0.19045.6466]

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\admin>mysql -u root -p
Enter password: ******
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 20
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+ ----------------- +
| Database          |
+ ----------------- +
| collegedb         |
| employee          |
| information_schema |
| mysql             |
| performance_schema |
| sys               |
+ ----------------- +
6 rows in set (0.03 sec)

mysql> use collegedb;
```

```
Database changed
mysql> show tables;
+ ------------------ +
| Tables_in_collegedb |
+ ------------------ +
| studentinfo         |
+ ------------------ +
1 row in set (0.01 sec)

mysql> select * from studentinfo;
+--------+----------+------+------------+ ------- +
| StudID | Sname    | Age  | Department | city   |
+--------+----------+------+------------+ ------- +
|    101 | Henry    |   20 | IT         | Mumbai |
|    102 | Maverick |   20 | IT         | Mumbai |
|    103 | Harsh    |   20 | COMPS      | Delhi  |
|    104 | Sameer   |   19 | EXTC       | Mumbai |
+--------+----------+------+------------+ ------- +
4 rows in set (0.01 sec)

mysql> create table Marks(SID int,Sname varchar(20),sub1 int,sub2 int,sub3 int);
Query OK, 0 rows affected (0.11 sec)

mysql> ALTER TABLE Marks Add column Total int;
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc Marks;
```

```
+-------+-------------+------+-----+---------+ ------ +
| Field | Type        | Null | Key | Default | Extra |
+-------+-------------+------+-----+---------+ ------ +
| SID   | int         | YES  |     | NULL    |       |
| Sname | varchar(20) | YES  |     | NULL    |       |
| sub1  | int         | YES  |     | NULL    |       |
| sub2  | int         | YES  |     | NULL    |       |
| sub3  | int         | YES  |     | NULL    |       |
| Total | int         | YES  |     | NULL    |       |
+-------+-------------+------+-----+---------+ ------ +
6 rows in set (0.02 sec)

mysql> insert into Marks Values(101,'Henry',45,38,44,NULL);
Query OK, 1 row affected (0.01 sec)

mysql> select * from Marks;
+------+-------+------+------+------+ ------ +
| SID  | Sname | sub1 | sub2 | sub3 | Total |
+------+-------+------+------+------+ ------ +
|  101 | Henry |   45 |   38 |   44 |  NULL |
+------+-------+------+------+------+ ------ +
1 row in set (0.00 sec)

mysql> Delimiter $$
mysql> create trigger Totalmarks
    -> before insert
    -> on Marks
    -> for each row
```

```
    -> begin
    -> set new.Total=new.sub1+new.sub2+new.sub3;
    -> end
    -> $$
Query OK, 0 rows affected (0.02 sec)

mysql> insert into Marks Values(102,'Maverick',43,39,44,NULL);
    -> $$
Query OK, 1 row affected (0.01 sec)

mysql> delimiter ;
mysql> insert into Marks Values(103,'Harsh',28,39,40,NULL);
Query OK, 1 row affected (0.01 sec)

mysql> select * from Marks;
+------+----------+------+------+------+-------+
| SID  | Sname    | sub1 | sub2 | sub3 | Total |
+------+----------+------+------+------+-------+
|  101 | Henry    |   45 |   38 |   44 |  NULL |
|  102 | Maverick |   43 |   39 |   44 |   126 |
|  103 | Harsh    |   28 |   39 |   40 |   107 |
+------+----------+------+------+------+-------+
3 rows in set (0.00 sec)

mysql> create table Update_log(log_id int auto_increment,StudID int,Old_age int,
New_age Int);
ERROR 1075 (42000): Incorrect table definition; there can be only one auto column
and it must be defined as a key
```

```
mysql> create table Update_log(log_id int auto_increment Primary key,StudID
int,Old_age int, New_age Int);

Query OK, 0 rows affected (0.04 sec)


mysql> delimiter $$

mysql> create trigger Age_update_trigger

    -> after update on studentinfo for each row

    -> begin

    -> if new.age>old.age then

    -> insert into Update_log(StudID,Old_age,New_age)
values(Old.StudID,Old.Age,New.Age);

    -> end if;

    -> end

    -> $$

Query OK, 0 rows affected (0.02 sec)


mysql> update studinfo set age =21 where StudID=103;

    -> $$

ERROR 1146 (42S02): Table 'collegedb.studinfo' doesn't exist

mysql> update studentinfo set age =21 where StudID=103$$

Query OK, 1 row affected (0.02 sec)

Rows matched: 1  Changed: 1  Warnings: 0


mysql> delimiter ;

mysql> select * from Update_log;

+--------+--------+---------+---------+

| log_id | StudID | Old_age | New_age |

+--------+--------+---------+---------+
```

```
|     1 |    103 |      20 |      21 |
+--------+---------+----------+----------+
1 row in set (0.00 sec)

mysql> select * from studentinfo;
+--------+----------+------+------------+ ------- +
| StudID | Sname    | Age  | Department | city   |
+--------+----------+------+------------+ ------- +
|    101 | Henry    |   20 | IT         | Mumbai |
|    102 | Maverick |   20 | IT         | Mumbai |
|    103 | Harsh    |   21 | COMPS      | Delhi  |
|    104 | Sameer   |   19 | EXTC       | Mumbai |
+--------+----------+------+------------+ ------- +
4 rows in set (0.00 sec)

mysql> use employee;
Database changed
mysql> select * from empinfo;
+-------+---------+---------+--------+-------- +
| EmpID | EmpName | Dept    | Salary | city    |
+-------+---------+---------+--------+-------- +
|     1 | Rohan   | IT      |  45000 | Delhi   |
|     2 | Simran  | HR      |  35000 | Mumbai  |
|     3 | Amit    | IT      |  60000 | Delhi   |
|     4 | Priya   | Finance |  55000 | Pune    |
|     5 | Karan   | Finance |  70000 | Delhi   |
|     6 | Sara    | IT      |  30000 | Mumbai  |
|     7 | Rahul   | IT      |  40000 | Chennai |
```

```
+-------+---------+---------+--------+-------- +
7 rows in set (0.00 sec)

mysql> create Table emp_delete_log(log_id int auto_increment primary key,EmpID
int, EmpName varchar(20));
Query OK, 0 rows affected (0.03 sec)

mysql> delimiter $$
mysql> create trigger del_employee
    -> before delete on empinfo for each row
    -> begin
    -> insert into emp_delete_log(EmpID,EmpName) values(old.EmpID,old.EmpName);
    -> end
    -> $$
Query OK, 0 rows affected (0.02 sec)

mysql> delimiter ;
mysql> delete from empinfo where EmpID=7;
Query OK, 1 row affected (0.01 sec)

mysql> select * from emp_delete_log;
+--------+-------+--------+
| log_id | EmpID | EmpName |
+--------+-------+--------+
|      1 |     7 | Rahul   |
+--------+-------+--------+
1 row in set (0.00 sec)
```

```
mysql> select * from empinfo;
+-------+---------+---------+--------+------- +
| EmpID | EmpName | Dept    | Salary | city   |
+-------+---------+---------+--------+------- +
|     1 | Rohan   | IT      |  45000 | Delhi  |
|     2 | Simran  | HR      |  35000 | Mumbai |
|     3 | Amit    | IT      |  60000 | Delhi  |
|     4 | Priya   | Finance |  55000 | Pune   |
|     5 | Karan   | Finance |  70000 | Delhi  |
|     6 | Sara    | IT      |  30000 | Mumbai |
+-------+---------+---------+--------+------- +
6 rows in set (0.00 sec)

mysql>
```

CONCLUSION:

In this experiment, we created multiple triggers to automate tasks such as calculating total marks, recording old and new age values, and logging deleted employee records. The experiment demonstrated that triggers are powerful tools for enforcing rules, maintaining consistency, and automatically responding to changes in a table. Overall, triggers help improve data accuracy and automate important database operations.

EXPERIMENT 7

AIM: To study and implement stored procedures in MySQL

THEORY:

A stored procedure is a precompiled block of SQL statements stored in the database and executed by calling its name. It allows grouping multiple SQL operations into a single reusable unit. Stored procedures improve performance, reduce code repetition, and enhance security by controlling data access.

Stored procedures are used when we need to perform repetitive tasks, implement business logic, validate data, update or fetch information, or reduce client-side processing. They also help in improving execution speed because SQL statements execute on the server side.

Syntax of a Stored Procedure:

```
CREATE PROCEDURE procedure_name(parameters)
BEGIN
SQL statements;
END;
```

Stored procedures may contain three types of parameters:

- IN: input values
- OUT: output values
- INOUT: used for both input and output

EXECUTION:

```
Microsoft Windows [Version 10.0.19045.6466]

(c) Microsoft Corporation. All rights reserved.


C:\Users\admin>mysql -u root -p

Enter password: ******

Welcome to the MySQL monitor.  Commands end with ; or \g.

Your MySQL connection id is 23
```

```
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use employee;
Database changed

mysql> select * from department;
+--------+-----------+ ----------- +
| DeptID | DeptName  | Manager     |
+--------+-----------+ ----------- +
|      1 | IT        | Mr.Arjun    |
|      2 | HR        | Ms.Kavita   |
|      3 | Finance   | Mr.Suresh   |
|      4 | Marketing | Ms.Natasha  |
|      5 | Sales     | Mr.Raghav   |
+--------+-----------+ ----------- +
5 rows in set (0.00 sec)

mysql> delimiter $$
mysql> CREATE PROCEDURE UpdateDeptManager( IN DID INT, IN
manager VARCHAR(20) )
    -> begin
```

```
    -> UPDATE department SET Manager = manager WHERE DeptID = DID;
    -> END
    -> $$
Query OK, 0 rows affected (0.01 sec)

mysql> call UpdateDeptManager(2,'Mr.Rohan');
    -> $$
Query OK, 1 row affected (0.01 sec)

mysql> select * from department;
    -> $$
+--------+-----------+ ----------- +
| DeptID | DeptName  | Manager     |
+--------+-----------+ ----------- +
|      1 | IT        | Mr.Arjun    |
|      2 | HR        | Mr.Rohan    |
|      3 | Finance   | Mr.Suresh   |
|      4 | Marketing | Ms.Natasha  |
|      5 | Sales     | Mr.Raghav   |
+--------+-----------+ ----------- +
5 rows in set (0.00 sec)

mysql> select * from empinfo;
    -> $$
+-------+---------+---------+--------+-------- +
| EmpID | EmpName | Dept    | Salary | city    |
+-------+---------+---------+--------+-------- +
|     1 | Rohan   | IT      |  45000 | Delhi   |
|     2 | Simran  | HR      |  35000 | Mumbai  |
```

```
|     3 | Amit     | IT       |  60000 | Delhi  |
|     4 | Priya    | Finance |  55000 | Pune    |
|     5 | Karan    | Finance |  70000 | Delhi  |
|     6 | Sara     | IT       |  30000 | Mumbai |
+-------+---------+---------+--------+--------+
6 rows in set (0.00 sec)

mysql> CREATE PROCEDURE CountEmpByDept( IN p_dept VARCHAR(50),
OUT emp_count INT )
    -> BEGIN
    -> SELECT COUNT(*) INTO emp_count FROM empinfo WHERE Dept =
p_dept;
    -> END
    -> $$
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
mysql> call CountEmpBYDept('IT',@c);
Query OK, 1 row affected (0.01 sec)

mysql> select @c;
+-----+
| @c   |
+-----+
|    3 |
+-----+
1 row in set (0.00 sec)

mysql>
```

CONCLUSION:

In this experiment, stored procedures were successfully created and executed to update the manager of a department and to count the number of employees in a specific department. This demonstrated how procedures simplify repeated operations, improve performance, and allow server-side execution of logic. Overall, stored procedures are powerful tools for structuring complex tasks and ensuring efficient and consistent database operations.

EXPERIMENT 8

AIM: To study and implement SQL Views by creating virtual tables using the CREATE VIEW command and performing operations such as selecting, updating, and modifying data through views.

THEORY: A View in SQL is a virtual table created using a SELECT query.
Unlike a physical table, a view does not store data itself. Instead, it displays data stored in one or more base tables. Views help simplify complex queries, enhance security by restricting direct table access, and present customized data according to user needs.

1. Horizontal View

A horizontal view restricts the output to specific **rows** of a table based on a given condition.

Characteristics:

- Filters rows using WHERE
- Shows all columns for the selected records
- Useful for displaying department-wise or region-wise data

Example:

CREATE VIEW View_ITEmployees AS SELECT * FROM Employee WHERE Department='IT';

2. Vertical View

A vertical view displays only selected **columns** of a table.

Characteristics:

- Shows specific columns only
- Includes all rows
- Used to hide unnecessary or sensitive fields

Example:

CREATE VIEW View_EmpNames AS SELECT EmpID,EmpName FROM Employee;

3. Row and Column Subset View

This view combines both row filtering and column selection.

Characteristics:

- Shows selected columns
- Only for rows matching a condition
- Useful for specific reporting tasks

Example:

CREATE VIEW View_SalesSummary AS SELECT EmpID,EmpName FROM Employee WHERE Department='Sales';

4. Group-Based View (Aggregate View)

A group view uses **GROUP BY** and aggregate functions such as SUM, AVG, COUNT, etc.

Characteristics:

- Contains grouped results
- Includes aggregated calculations
- Usually **not updatable**
- Useful for department-wise summaries or statistics

Example:

CREATE VIEW View_DeptStats AS SELECT Department,COUNT(EmpID) AS TotalEmp,AVG(Salary) AS AvgSal FROM Employee GROUP BY Department;

5. Join-Based View

A join view is created using data combined from **two or more tables** through JOIN operations.

Characteristics:

- Displays related data from multiple tables
- Useful for showing merged information (employee + department details)

- Often not updatable when multiple tables are involved

Example:

CREATE VIEW View_EmpDept AS SELECT e.EmpID,e.EmpName,e.Sala

EXECUTION:

Microsoft Windows [Version 10.0.19045.6466]

(c) Microsoft Corporation. All rights reserved.

C:\Users\admin>mysql -u root -p

Enter password: ******

Welcome to the MySQL monitor. Commands end with ; or \g.

Your MySQL connection id is 19

Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> use Employee;
Database changed
mysql> select * from empinfo;
+-------+---------+---------+--------+-------- +
| EmpID | EmpName | Dept    | Salary | city    |
+-------+---------+---------+--------+-------- +
```

```
|     1 | Rohan   | IT      |  45000 | Delhi   |
|     2 | Simran  | HR      |  35000 | Mumbai  |
|     3 | Amit    | IT      |  60000 | Delhi   |
|     4 | Priya   | Finance |  55000 | Pune    |
|     5 | Karan   | Finance |  70000 | Delhi   |
|     6 | Sara    | IT      |  30000 | Mumbai  |
|     7 | Rahul   | IT      |  40000 | Chennai |
+-------+---------+---------+--------+-------- +
7 rows in set (0.01 sec)

mysql> CREATE VIEW IT_Dept_Emp AS SELECT * FROM empinfo WHERE Dept = 'IT';
Query OK, 0 rows affected (0.07 sec)

mysql> select * from IT_Dept_Emp;
+-------+---------+------+--------+----------+
| EmpID | EmpName | Dept | Salary | city     |
+-------+---------+------+--------+----------+
|     1 | Rohan   | IT   |  45000 | Delhi    |
|     3 | Amit    | IT   |  60000 | Delhi    |
|     6 | Sara    | IT   |  30000 | Mumbai   |
|     7 | Rahul   | IT   |  40000 | Chennai  |
+-------+---------+------+--------+----------+
4 rows in set (0.01 sec)

mysql> CREATE VIEW emp_basic_info AS SELECT EmpID, EmpName, Dept FROM
empinfo;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> select * from emp_basic_info;
+-------+---------+--------+
| EmpID | EmpName | Dept   |
+-------+---------+--------+
|     1 | Rohan   | IT     |
|     2 | Simran  | HR     |
|     3 | Amit    | IT     |
|     4 | Priya   | Finance |
|     5 | Karan   | Finance |
|     6 | Sara    | IT     |
|     7 | Rahul   | IT     |
+-------+---------+--------+
7 rows in set (0.00 sec)
```

```
mysql> CREATE VIEW IT_Dept_Emp_basic_info AS SELECT EmpID, EmpName, Salary
FROM empinfo WHERE Dept = 'IT';
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> select * from IT_Dept_Emp_basic_info;
+-------+---------+-------+
| EmpID | EmpName | Salary |
+-------+---------+-------+
|     1 | Rohan   |  45000 |
|     3 | Amit    |  60000 |
|     6 | Sara    |  30000 |
|     7 | Rahul   |  40000 |
+-------+---------+-------+
4 rows in set (0.00 sec)
```

```
mysql> CREATE VIEW emp_dept_summary AS SELECT Dept, COUNT(*) AS EmpCount,
AVG(Salary) AS AvgSalary FROM empinfo GROUP BY Dept;

Query OK, 0 rows affected (0.01 sec)


mysql> select * from emp_dept_summary;

+---------+----------+ ---------- +

| Dept    | EmpCount | AvgSalary  |

+---------+----------+ ---------- +

| IT      |        4 | 43750.0000 |

| HR      |        1 | 35000.0000 |

| Finance |        2 | 62500.0000 |

+---------+----------+ ---------- +

3 rows in set (0.01 sec)


mysql> show tables

    -> show tables;

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near 'show
tables' at line 2

mysql> show tables;

+ ---------------------+

| Tables_in_employee     |

+ ---------------------+

| department             |

| emp_basic_info         |

| emp_dept_summary       |

| empinfo                |

| it_dept_emp            |
```

```
| it_dept_emp_basic_info |
+ ---------------------+
6 rows in set (0.01 sec)

mysql> select * from department;
+--------+-----------+ ----------- +
| DeptID | DeptName  | Manager     |
+--------+-----------+ ----------- +
|      1 | IT        | Mr. Arjun   |
|      2 | HR        | Ms. Kavita  |
|      3 | Finance   | Mr. Suresh  |
|      4 | Marketing | Ms. Natasha |
|      5 | Sales     | Mr. Raghav  |
+--------+-----------+ ----------- +
5 rows in set (0.00 sec)

mysql> CREATE VIEW v_emp_with_manager AS SELECT e.EmpID, e.EmpName,
e.Dept, e.Salary, e.city, d.Manager FROM empinfo e LEFT JOIN department d ON
e.Dept = d.DeptName;
Query OK, 0 rows affected (0.01 sec)

mysql> select * from v_emp_with_manager;
+-------+---------+---------+--------+---------+ ----------- +
| EmpID | EmpName | Dept    | Salary | city    | Manager     |
+-------+---------+---------+--------+---------+ ----------- +
|     1 | Rohan   | IT      |  45000 | Delhi   | Mr. Arjun   |
|     2 | Simran  | HR      |  35000 | Mumbai  | Ms. Kavita  |
|     3 | Amit    | IT      |  60000 | Delhi   | Mr. Arjun   |
```

```
|    4 | Priya   | Finance |  55000 | Pune     | Mr. Suresh |
|    5 | Karan   | Finance |  70000 | Delhi    | Mr. Suresh |
|    6 | Sara    | IT      |  30000 | Mumbai   | Mr. Arjun  |
|    7 | Rahul   | IT      |  40000 | Chennai  | Mr. Arjun  |
+-------+---------+---------+--------+---------+ ----------- +
7 rows in set (0.01 sec)

mysql>
```

CONCLUSION:

In this experiment, SQL Views were created and executed to understand how virtual tables are built using SELECT queries. The experiment demonstrated how views simplify complex queries, provide secure access to selected columns, and present data in a customized format. Overall, this experiment strengthened the understanding of how views improve data management, accessibility, and modularity in SQL.

EXPERIMENT 9

AIM: To implement CRUD (Create, Read, Update, Delete) operations using Java Database Connectivity (JDBC).

THEORY:

1. CRUD Operations: CRUD stands for the four basic operations of persistent storage:

- **Create (INSERT):** Adds new records to the database.
- **Read (SELECT):** Retrieves data from the database.
- **Update (UPDATE):** Modifies existing records.
- **Delete (DELETE):** Removes records from the database.

2. PreparedStatement: In JDBC, PreparedStatement is used to execute parameterized SQL queries. It is preferred over Statement for CRUD operations because:

- It prevents SQL Injection attacks.
- It improves performance via pre-compilation.
- It allows dynamic values to be set using setters like setString(), setInt().

EXECUTION :

1. CREATE OPERATION (Insert Data)

This program inserts a new OTT subscription record into the database.

Code (INSERTP.java):

Java
```
import java.sql.*;
import java.util.Scanner;

class INSERTP {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      try {
         // Load Driver
         Class.forName("com.mysql.cj.jdbc.Driver");
```

```
        // Establish Connection
        Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/OttDB", "root",
"oracle");

        // Prepare Query
        String sql = "INSERT INTO Ott_sub(ott, mon_sub, 6mon_sub, year_sub)
VALUES (? , ? , ?, ?)";
        PreparedStatement ps = conn.prepareStatement(sql);

        // Input from user
        System.out.println("enter ott :");
        String ott = sc.nextLine();
        System.out.println("enter mon_sub :");
        String mon_sub = sc.nextLine();
        System.out.println("enter 6mon_sub :");
        String mon6_sub = sc.nextLine();
        System.out.println("enter year_sub :");
        String year_sub = sc.nextLine();

        // Passing values to Query
        ps.setString(1, ott);
        ps.setString(2, mon_sub);
        ps.setString(3, mon6_sub);
        ps.setString(4, year_sub);

        // Execute Update
        int rowsInserted = ps.executeUpdate();
        if (rowsInserted > 0) {
            System.out.println("Record inserted successfully!");
        }
        conn.close();
    } catch (Exception e) {
        System.out.println(e);
    }
  }
}
```

Output:

```
enter ott :
Netflix
enter mon_sub :
199
```

```
enter 6mon_sub :
1194
enter year_sub :
2388
Record inserted successfully!
```

2. READ OPERATION (Select Data)

This program fetches and displays all available OTT subscriptions.

Code (MSConn.java):

Java
```
import java.sql.*;

class MSConn {
   public static void main(String args[]) {
      try {
         Class.forName("com.mysql.cj.jdbc.Driver");
         Connection con =
DriverManager.getConnection("jdbc:mysql://localhost/OttDB", "root", "oracle");
         Statement stmt = con.createStatement();

         // Execute Select Query
         ResultSet rs = stmt.executeQuery("select * from Ott_sub");

         System.out.println("Ott \t | mon_sub | 6mon_sub | year_sub");
         System.out.println(" ---------------------------------------- ");
         while (rs.next()) {
            System.out.println(rs.getString(1) + "\t" + rs.getString(2) + "\t" +
rs.getString(3) + "\t" + rs.getString(4));
         }
         con.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

Output:

```
Ott         | mon_sub | 6mon_sub | year_sub
- - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Netflix     199        1194       2388
```

Prime 179 999 1499

3. UPDATE OPERATION

This program updates the monthly subscription price for a specific OTT plaVorm.

Code (UPDATEP.java):

Java
```
import java.sql.*;
import java.util.Scanner;

class UPDATEP {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      try {
         Class.forName("com.mysql.cj.jdbc.Driver");
         Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/OttDB", "root",
"oracle");

         String sql = "UPDATE Ott_sub SET mon_sub = ? WHERE ott = ?";
         PreparedStatement ps = conn.prepareStatement(sql);

         System.out.println("Enter OTT name to update:");
         String ott = sc.nextLine();
         System.out.println("Enter new monthly price:");
         String new_price = sc.nextLine();

         ps.setString(1, new_price);
         ps.setString(2, ott);

         int rowsUpdated = ps.executeUpdate();
         if (rowsUpdated > 0) {
            System.out.println("Record updated successfully!");
         } else {
            System.out.println("No record found to update.");
         }
         conn.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

Output:

Enter OTT name to update:
Netflix
Enter new monthly price:
249
Record updated successfully!

4. DELETE OPERATION

This program deletes a record from the database based on the OTT name.

Code (DELETEP.java):

Java
```
import java.sql.*;
import java.util.Scanner;

class DELETEP {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      try {
         Class.forName("com.mysql.cj.jdbc.Driver");
         Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/OttDB", "root",
"oracle");

         String sql = "DELETE FROM Ott_sub WHERE ott = ?";
         PreparedStatement ps = conn.prepareStatement(sql);

         System.out.println("Enter OTT name to delete:");
         String ott = sc.nextLine();

         ps.setString(1, ott);

         int rowsDeleted = ps.executeUpdate();
         if (rowsDeleted > 0) {
            System.out.println("Record deleted successfully!");
         } else {
            System.out.println("No record found to delete.");
         }
         conn.close();
      } catch (Exception e) {
         System.out.println(e);
```

```
        }
    }
}
```

Output:

Enter OTT name to delete:
Prime
Record deleted successfully!

CONCLUSION:

Thus, we have successfully implemented all four CRUD operations (Create, Read, Update, Delete) on a MySQL database using JDBC in Java.

EXPERIMENT 10

AIM: To study SQL subqueries (nested queries), their types, execution behavior and to implement examples

THEORY:

Subqueries, also known as nested queries, are SQL queries placed inside another query to derive intermediate results that help filter, compare, or group data. The inner query executes first, and its output is used by the outer query. Subqueries are useful when a condition depends on values that must be computed dynamically, such as averages, minimum or maximum values, or specific sets of data obtained from other tables.

The commands executed above demonstrate **different types of subqueries**, each serving a unique purpose in data retrieval and analysis.

1. Single-Row (Scalar) Subquery

A single-row subquery returns exactly one value. It is used with comparison operators such as =, >, <, >=, etc.

Theory:
This query calculates the average salary using the inner subquery, then retrieves all employees whose salaries are greater than that average. Since the subquery returns a single numeric value (the average), it is a scalar subquery.

2. Multi-Row Subquery using IN

A multi-row subquery returns multiple rows and is used with operators like IN, ANY, or ALL.

Theory:
The inner subquery returns all department IDs located in Mumbai or Pune. The outer query selects all staff who belong to these departments. Since the subquery returns multiple values, the IN operator is used.

3. NOT IN Subquery

This retrieves records that **do not match** any value returned by the subquery.

Theory:
The inner subquery finds all department IDs that currently have staff assigned.
The outer query retrieves only those departments **with no staff**, demonstrating the use of NOT IN to filter unmatched rows.

4. Aggregate Subquery (Single-Row)

A subquery that uses aggregate functions (like MIN, MAX, SUM, AVG) returns a single value.

Theory:
The subquery finds the minimum salary, and the outer query retrieves the employee who earns that minimum salary. This is a scalar aggregate subquery.

5. Multi-Row Subquery with Location Filtering

This example finds staff members based on department locations.

Theory:
The inner query identifies departments located in Delhi. The outer query returns only those staff members whose department ID matches the returned results.

<u>EXECUTION:</u>

```
Microsoft Windows [Version 10.0.19045.6466]

(c) Microsoft Corporation. All rights reserved.


C:\Users\admin>mysql -u root -p

Enter password: ******

Welcome to the MySQL monitor.  Commands end with ; or \g.

Your MySQL connection id is 16

Server version: 8.0.44 MySQL Community Server - GPL
```

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> show databases;
+ ----------------- +
| Database          |
+ ----------------- +
| collegedb         |
| employee          |
| information_schema |
| mysql             |
| performance_schema |
| sys               |
+ ----------------- +
6 rows in set (0.01 sec)

mysql> use employee;
Database changed
mysql> show tables;
+ ----------------- +
| Tables_in_employee |
+ ----------------- +
| department        |
```

```
| empinfo          |
+ ----------------- +
2 rows in set (0.01 sec)

mysql> select * from empinfo;
+-------+---------+---------+--------+------- +
| EmpID | EmpName | Dept    | Salary | city   |
+-------+---------+---------+--------+------- +
|     1 | Rohan   | IT      |  45000 | Delhi  |
|     2 | Simran  | HR      |  35000 | Mumbai |
|     3 | Amit    | IT      |  60000 | Delhi  |
|     4 | Priya   | Finance |  55000 | Pune   |
|     5 | Karan   | Finance |  70000 | Delhi  |
|     6 | Sara    | IT      |  30000 | Mumbai |
+-------+---------+---------+--------+------- +
6 rows in set (0.00 sec)

mysql> SELECT EmpName, Salary FROM empinfo WHERE Salary > (SELECT
AVG(Salary) FROM empinfo);
+---------+-------+
| EmpName | Salary |
+---------+-------+
| Amit    |  60000 |
| Priya   |  55000 |
| Karan   |  70000 |
+---------+-------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT EmpName, Salary FROM empinfo WHERE Salary < ( SELECT
MIN(Salary) FROM empinfo WHERE Dept = 'IT' );

Empty set (0.02 sec)


mysql> SELECT EmpName, Dept FROM empinfo WHERE Dept = ( SELECT Dept FROM
empinfo WHERE EmpName = 'Priya' );

+---------+--------+

| EmpName | Dept    |

+---------+--------+

| Priya   | Finance |

| Karan   | Finance |

+---------+--------+

2 rows in set (0.00 sec)


mysql> SELECT EmpName, Salary FROM empinfo WHERE Salary = (SELECT
MAX(Salary) FROM empinfo);

+---------+-------+

| EmpName | Salary |

+---------+-------+

| Karan   |  70000 |

+---------+-------+

1 row in set (0.01 sec)


mysql> SELECT EmpName, City FROM empinfo WHERE City = ( SELECT City FROM
empinfo WHERE Salary = (SELECT MAX(Salary) FROM empinfo) );

+---------+------+

| EmpName | City  |

+---------+------+

| Rohan   | Delhi |
```

```
| Amit     | Delhi |
| Karan    | Delhi |
+---------+------ +
3 rows in set (0.01 sec)

mysql>
```

CONCLUSION:

Through the executed queries, different types of SQL subqueries were demonstrated, including scalar, multi-row, NOT IN, and aggregate subqueries. These subqueries allowed filtering data based on computed values such as averages and minimum salaries, as well as retrieving records dependent on other tables such as department locations.

EXPERIMENT 11

AIM: To study and implement SQL Transaction Control Language (TCL) commands such as START TRANSACTION, SAVEPOINT, ROLLBACK, and COMMIT, and to observe how they control changes made to table data during a transaction.

THEORY:

Transaction Control Language (TCL) commands in SQL are used to manage changes made by DML operations. A transaction is a sequence of operations performed as a single logical unit of work. TCL ensures that these modifications are either completely saved or completely undone, maintaining the integrity and reliability of the database.

The commands demonstrated in the code were:

1. START TRANSACTION

This command begins a new transaction block.
All changes made after this command remain temporary until they are committed.

2. SAVEPOINT

A SAVEPOINT is a checkpoint inside a transaction.
It allows partial rollbacks to specific stages instead of undoing the entire transaction.

Each SAVEPOINT marked a safe state that you could return to.

3. ROLLBACK / ROLLBACK TO SAVEPOINT

The ROLLBACK command undoes the changes made in the transaction.

Rollback ensures:

- No accidental update or deletion becomes permanent
- The table returns to a consistent state

4. COMMIT

The COMMIT command makes all the changes permanent.

Once committed, changes cannot be rolled back.

EXECUTION:

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin>mysql -u root -p
Enter password: ******
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 18
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use Employee;
Database changed
mysql> show tables;
+ ----------------- +
| Tables_in_employee |
+ ----------------- +
```

```
| department        |
| empinfo           |
+ ----------------- +
2 rows in set (0.07 sec)

mysql> select * from empinfo;
+-------+---------+---------+--------+------- +
| EmpID | EmpName | Dept    | Salary | city   |
+-------+---------+---------+--------+------- +
|     1 | Rohan   | IT      |  45000 | Delhi  |
|     2 | Simran  | HR      |  35000 | Mumbai |
|     3 | Amit    | IT      |  60000 | Delhi  |
|     4 | Priya   | Finance |  55000 | Pune   |
|     5 | Karan   | Finance |  70000 | Delhi  |
|     6 | Sara    | IT      |  30000 | Mumbai |
+-------+---------+---------+--------+------- +
6 rows in set (0.00 sec)

mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into empinfo values(7,'Rahul','IT',40000,'Chennai');
Query OK, 1 row affected (0.04 sec)

mysql> savepoint A;
Query OK, 0 rows affected (0.01 sec)

mysql> update empinfo set Salary= 65000 where EmpID=3;
```

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> savepoint B;
Query OK, 0 rows affected (0.00 sec)

mysql> Delete from empinfo where EmpID=6;
Query OK, 1 row affected (0.01 sec)

mysql> savepoint C;
Query OK, 0 rows affected (0.00 sec)

mysql> Rollback to C;
Query OK, 0 rows affected (0.01 sec)

mysql> select * from empinfo;
+-------+---------+---------+--------+-------- +
| EmpID | EmpName | Dept    | Salary | city    |
+-------+---------+---------+--------+-------- +
|     1 | Rohan   | IT      |  45000 | Delhi   |
|     2 | Simran  | HR      |  35000 | Mumbai  |
|     3 | Amit    | IT      |  65000 | Delhi   |
|     4 | Priya   | Finance |  55000 | Pune    |
|     5 | Karan   | Finance |  70000 | Delhi   |
|     7 | Rahul   | IT      |  40000 | Chennai |
+-------+---------+---------+--------+-------- +
6 rows in set (0.00 sec)
```

```
mysql> Rollback to B;
Query OK, 0 rows affected (0.01 sec)

mysql> select * from empinfo;
+-------+---------+---------+--------+-------- +
| EmpID | EmpName | Dept    | Salary | city    |
+-------+---------+---------+--------+-------- +
|     1 | Rohan   | IT      |  45000 | Delhi   |
|     2 | Simran  | HR      |  35000 | Mumbai  |
|     3 | Amit    | IT      |  65000 | Delhi   |
|     4 | Priya   | Finance |  55000 | Pune    |
|     5 | Karan   | Finance |  70000 | Delhi   |
|     6 | Sara    | IT      |  30000 | Mumbai  |
|     7 | Rahul   | IT      |  40000 | Chennai |
+-------+---------+---------+--------+-------- +
7 rows in set (0.00 sec)

mysql> Rollback to A;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from empinfo;
+-------+---------+---------+--------+-------- +
| EmpID | EmpName | Dept    | Salary | city    |
+-------+---------+---------+--------+-------- +
|     1 | Rohan   | IT      |  45000 | Delhi   |
|     2 | Simran  | HR      |  35000 | Mumbai  |
|     3 | Amit    | IT      |  60000 | Delhi   |
|     4 | Priya   | Finance |  55000 | Pune    |
```

```
|     5 | Karan   | Finance |  70000 | Delhi   |
|     6 | Sara    | IT      |  30000 | Mumbai  |
|     7 | Rahul   | IT      |  40000 | Chennai |
+-------+---------+---------+--------+-------- +
7 rows in set (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.01 sec)

mysql> select * from empinfo;
+-------+---------+---------+--------+-------- +
| EmpID | EmpName | Dept    | Salary | city    |
+-------+---------+---------+--------+-------- +
|     1 | Rohan   | IT      |  45000 | Delhi   |
|     2 | Simran  | HR      |  35000 | Mumbai  |
|     3 | Amit    | IT      |  60000 | Delhi   |
|     4 | Priya   | Finance |  55000 | Pune    |
|     5 | Karan   | Finance |  70000 | Delhi   |
|     6 | Sara    | IT      |  30000 | Mumbai  |
|     7 | Rahul   | IT      |  40000 | Chennai |
+-------+---------+---------+--------+-------- +
7 rows in set (0.00 sec)
mysql>
```

CONCLUSION:

In this experiment, we successfully demonstrated how MySQL transaction control commands allow safe and controlled manipulation of data. By using START TRANSACTION, SAVEPOINT, ROLLBACK, and COMMIT, we observed how changes can be grouped, tested, undone, or permanently saved.

Savepoints allowed us to partially roll back to any intermediate stage without affecting the complete transaction. Rolling back to C restored the table to the state before the delete operation, rolling back to B restored the state before the delete but retained the update, and rolling back to A reversed both update and delete while keeping only the insertion. Finally, COMMIT permanently stored the final state of all operations.

This experiment shows that transaction control ensures reliability, consistency, and error recovery in databases, protecting data from accidental or incorrect changes.